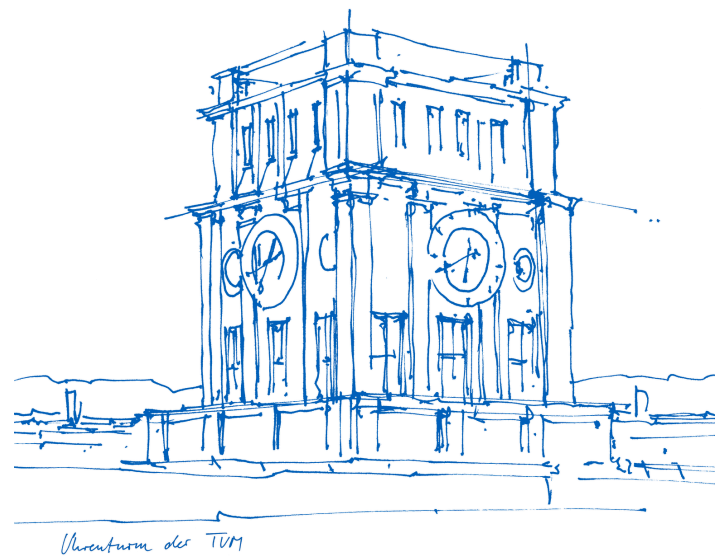
Statistical Learning and Data Analytics for Transportation Systems

Kernel Machines

Soban Lone
soban.lone@tum.de

Technical University of Munich
TUM School of Engineering and Design
Chair of Transportation Systems Engineering

Munich, 05/06/2026



Recap: Linear Foundations & The Path to Kernels

- **Linear Regression & Regularization**: Predicting continuous outputs while controlling model complexity (L1/L2).
- **Dimensionality Reduction**: Projecting complex, high-dimensional data onto major axes of variance.
- **Time Series**: Extracting sequential patterns and tracking temporal dependencies.
- **Linear Classification**: Partitioning feature spaces using rigid, straight-line boundaries.

The Motivation for Kernel Methods

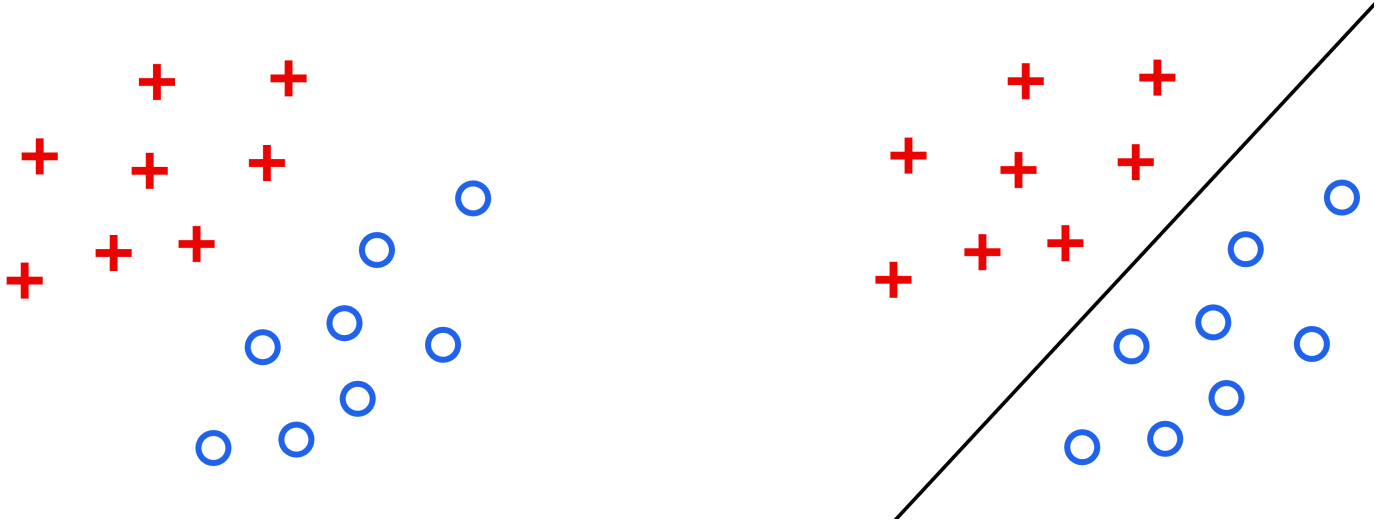
- **The Linear Limit**: Real-world data structures are rarely straight (e.g., circles, XOR).
- **The Kernel Trick**: Accomplishing this mapping implicitly by swapping dot products for similarity functions.

Linear SVM

Linearly separable data

Preliminaries

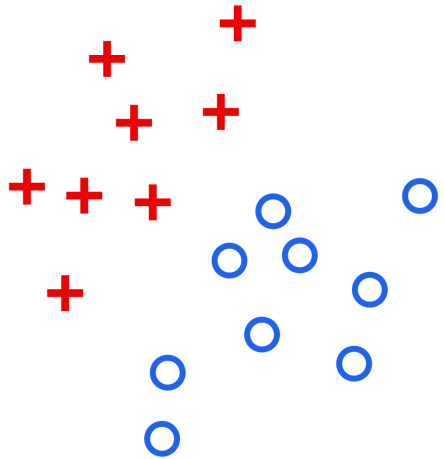
How will a decision boundary of a linear classifier look like?

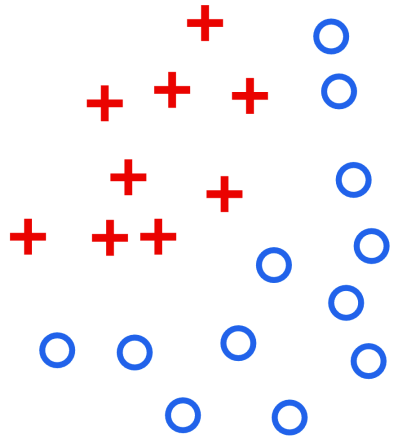


Linearly separable data

Preliminaries

What about these datasets?

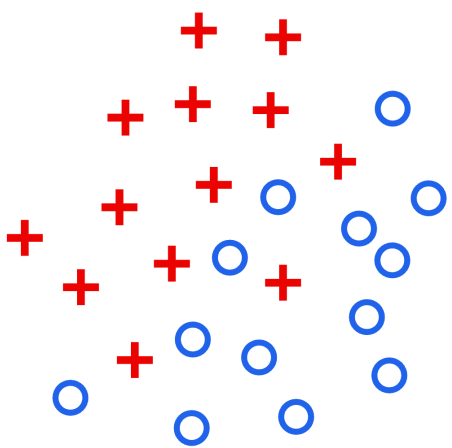


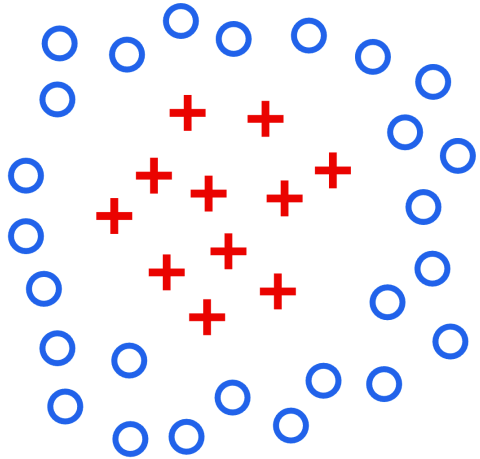


Linearly separable data

Preliminaries

What about these datasets?





Linearly separable data

Preliminaries

Linearly Separable

It's when two classes of data can be separated from each other by a **hyperplane**.

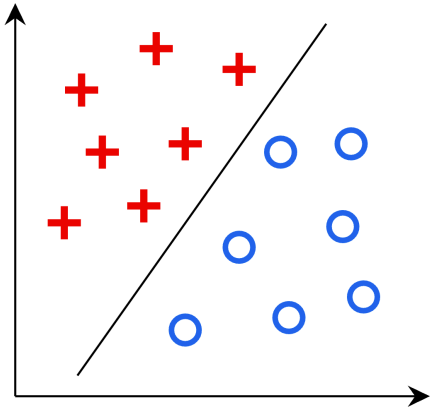
A **hyperplane** is a $p-1$-dimensional affine subspace of a p-dimensional space.

$$\boldsymbol{w}'\boldsymbol{x} + b = w_1 x_1 + w_2 x_2 + \cdots + w_p x_p + b = 0$$

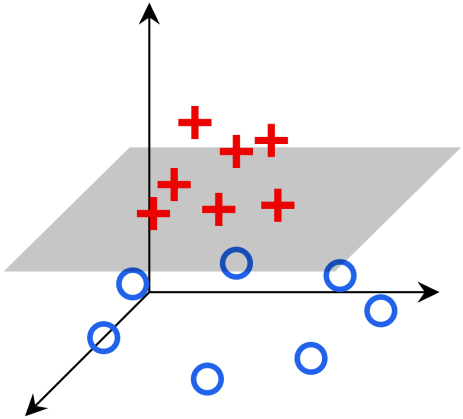
Linearly separable data

Preliminaries

1-dim



2-dim



3-dim

Linearly separable data

Support vector machine can make classifications using a separating **hyperplane**. It can perfectly separate linearly separable data.

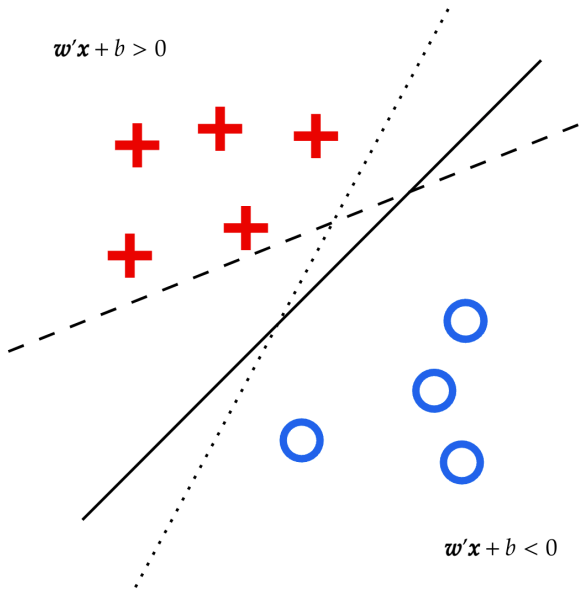
$$\begin{cases} y_i = +1, & \text{if } \boldsymbol{w}'\boldsymbol{x}_i + b > 0, \\ y_i = -1, & \text{if } \boldsymbol{w}'\boldsymbol{x}_i + b < 0 \end{cases}$$

$$\Downarrow$$

$$y_i = \operatorname{sgn}(\boldsymbol{w}'\boldsymbol{x}_i + b)$$

$$\Downarrow$$

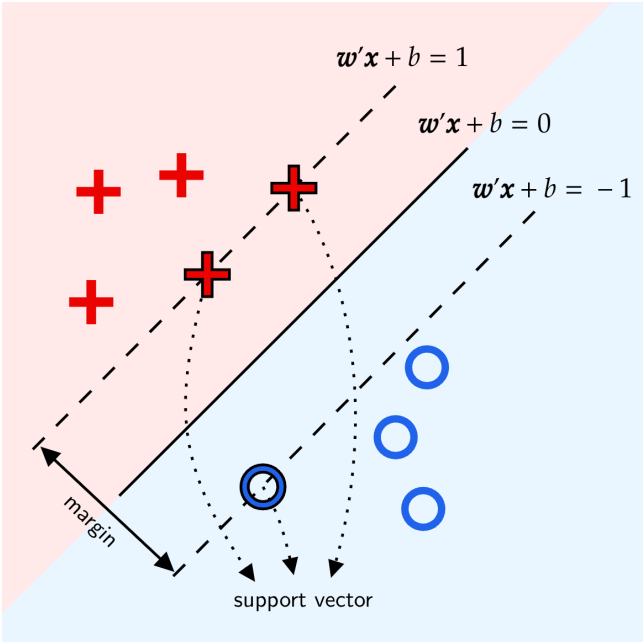
$$y_i(\boldsymbol{w}'\boldsymbol{x}_i + b) > 0$$



The central separating hyperplane is the decision boundary.

Margin & Support Vector

- For linearly separable data, we can always find **a margin** between samples of the two classes.
- **Support vectors** lie on the the boundary of the margin.
- The separating hyperplane is determined by the support vectors.



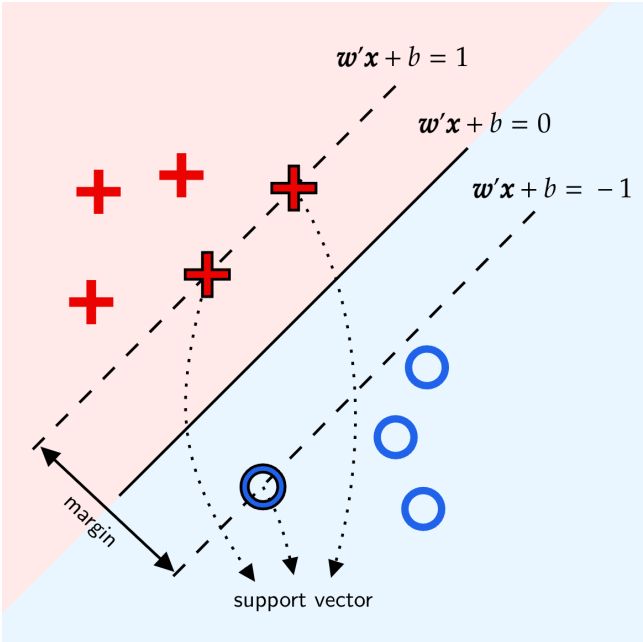
Margin & Support Vector

The distance from an arbitrary point $\boldsymbol{x}_0$ to the hyperplane $\boldsymbol{w}'\boldsymbol{x} + b = 0$ can be expressed as:

$$\text{dist}(\boldsymbol{x}_0 \| \boldsymbol{w}'\boldsymbol{x} + b = 0) = \frac{|\boldsymbol{w}'\boldsymbol{x}_0 + b|}{\|\boldsymbol{w}\|}$$

We can further obtain the width of margin:

$$\gamma = \frac{|1|}{\|\boldsymbol{w}\|} + \frac{|-1|}{\|\boldsymbol{w}\|} = \frac{2}{\|\boldsymbol{w}\|}$$



Linear SVM

- **Prerequisite**: Linearly separable data.
- **Objective**: Maximize the **width of margin** around separating hyperplane.

$$\max_{\boldsymbol{w},b} \frac{2}{\|\boldsymbol{w}\|}$$
$$\text{s.t.} \quad y_i(\boldsymbol{w}'\boldsymbol{x}_i + b) \geq 1, \quad i = 1, 2, \ldots, n$$

For computational convenience, the problem can be equivalently transformed into a **convex quadratic program** with linear constraints:

$$\min_{\boldsymbol{w},b} \frac{\|\boldsymbol{w}\|^2}{2}$$
$$\text{s.t.} \quad y_i(\boldsymbol{w}'\boldsymbol{x}_i + b) \geq 1, \quad i = 1, 2, \ldots, n$$

Dual problem of Linear SVM

How to deal with inequality constraints in convex optimization?

$$\min_{\boldsymbol{w},b} \frac{\|\boldsymbol{w}\|^2}{2}$$

$$\text{s.t.} \quad y_i(\boldsymbol{w}'\boldsymbol{x}_i + b) \geq 1, \quad i = 1, 2, \ldots, n$$

$$1 - y_i(\boldsymbol{w}'\boldsymbol{x}_i + b) \leq 0$$

Lagrange Multiplier Method : Transform a constrained problem by augmenting the objective function with a weighted sum of the constraint functions.

Lagrangian $L : \mathbb{R}^m \times \mathbb{R} \times \mathbb{R}^n \to \mathbb{R}$:

$$L(\boldsymbol{w}, b, \boldsymbol{\alpha}) = \frac{\|\boldsymbol{w}\|^2}{2} + \sum_{i=1}^{n} \alpha_i \left(1 - y_i \left(\boldsymbol{w}'\boldsymbol{x}_i + b\right)\right)$$

Lagrangian multiplier (non-negative)

Dual problem of Linear SVM

Primal problem:

$$\min_{\boldsymbol{w},b} \frac{\|\boldsymbol{w}\|^2}{2}$$
$$\text{s.t.} \quad 1 - y_i(\boldsymbol{w}'\boldsymbol{x}_i + b) \leq 0, \quad i = 1, 2, \ldots, n$$

Lagrangian:

$$L(\boldsymbol{w}, b, \boldsymbol{\alpha}) = \frac{\|\boldsymbol{w}\|^2}{2} + \sum_{i=1}^{n} \alpha_i \left(1 - y_i \left(\boldsymbol{w}'\boldsymbol{x}_i + b\right)\right)$$

Dual problem:

$$\max_{\boldsymbol{\alpha} \succeq 0} \min_{\boldsymbol{w},b} L(\boldsymbol{w}, b, \boldsymbol{\alpha})$$

Dual problem of Linear SVM

Gradient of Lagrangian:

$$\nabla_{\boldsymbol{w}} L(\boldsymbol{w}^*, b^*, \boldsymbol{\alpha}^*) = 0 \quad \Leftrightarrow \quad \boldsymbol{w}^* = \sum_{i=1}^{n} \alpha_i^* y_i \boldsymbol{x}_i$$

$$\nabla_{b} L(\boldsymbol{w}^*, b^*, \boldsymbol{\alpha}^*) = 0 \quad \Leftrightarrow \quad 0 = \sum_{i=1}^{n} \alpha_i^* y_i$$

Dual problem:

$$\begin{aligned} \max_{\boldsymbol{\alpha}} & \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \boldsymbol{x}_i' \boldsymbol{x}_j \\ \text{s.t.} & \sum_{i=1}^{n} \alpha_i y_i = 0, \\ & \alpha_i \geq 0, \quad i = 1, 2, \ldots, n \end{aligned}$$

Dual problem of Linear SVM

KKT conditions:

1. *Primal constraints*: $1 - y_i(\boldsymbol{w}^{*\prime}\boldsymbol{x}_i + b^*) \leq 0, \quad i = 1, 2, \ldots, n$
2. *Dual constraints*: $\alpha_i^* \geq 0, \quad i = 1, 2, \ldots, n$
3. *Complementary slackness*:

$$\alpha_i^* \left(1 - y_i(\boldsymbol{w}^{*\prime}\boldsymbol{x}_i + b^*)\right) = 0, \quad i = 1, 2, \ldots, n$$

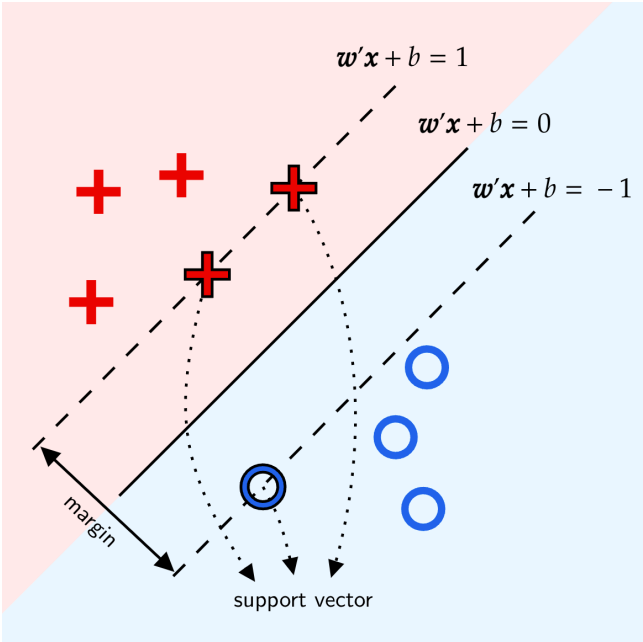
4. *Gradient of Lagrangian*:

$$\begin{aligned} \nabla_{\boldsymbol{w}} L(\boldsymbol{w}^*, b^*, \boldsymbol{\alpha}^*) &= 0 \\ \nabla_b L(\boldsymbol{w}^*, b^*, \boldsymbol{\alpha}^*) &= 0 \end{aligned}$$

Dual problem of Linear SVM

According to (3) complementary slackness, we have either

- $y_i(\boldsymbol{w}^{*\prime}\boldsymbol{x}_i + b^*) = 1$. This indicates that the i-th sample is a **support vector**.
- or $\alpha_i^* = 0$. This indicates the i-th sample is outside the margin.



Dual problem of Linear SVM

Assume the optimal solution of the dual problem is $\boldsymbol{\alpha}^*$. Using (4) gradient of Lagrangian, we've already obtained:

$$\boldsymbol{w}^* = \sum_{i=1}^{n} \alpha_i^* y_i \boldsymbol{x}_i$$

Denote an arbitrary support vector as $\boldsymbol{x}_s, y_s$. Using (3) complementary slackness, we can obtain:

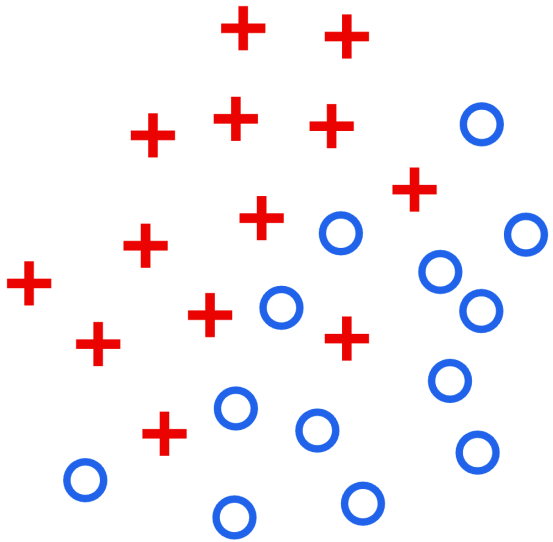
$$b^* = \frac{1}{y_s} - \sum_{i=1}^{n} \alpha_i^* y_i \boldsymbol{x}_i' \boldsymbol{x}_s = y_s - \sum_{i=1}^{n} \alpha_i^* y_i \boldsymbol{x}_i' \boldsymbol{x}_s$$

$\boldsymbol{w}^*$ and b^* are only determined by support vectors.

Soft Margin SVM

Non-Separable Data

Can linear SVM solve the following classification problem?



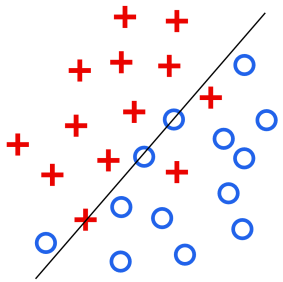
Non-Separable Data

The basic assumption of hard margin SVM is too rigorous.

- All the samples should lie outside the margin and be correctly classified, i.e., $y_i(\boldsymbol{w}'\boldsymbol{x}_i + b) \geq 1$.

This assumption can be relaxed as follows:

- Allow some samples to be classified incorrectly, i.e., $y_i(\boldsymbol{w}'\boldsymbol{x}_i + b) < 1$.



Soft Margin

The mathematical model of SVM can be changed as follows:

- Remove the old constraint $y_i(\boldsymbol{w}'\boldsymbol{x}_i + b) \geq 1$ to allow incorrect classifications.
- Add penalty $\ell\left(y_i(\boldsymbol{w}'\boldsymbol{x}_i + b)\right)$ to the objective function.

*We've turned the hard margin into a **soft margin**!*

$$\min_{\boldsymbol{w},b} \frac{\|\boldsymbol{w}\|^2}{2}$$

$$\text{s.t.} \quad y_i(\boldsymbol{w}'\boldsymbol{x}_i + b) \geq 1, \quad i = 1, 2, \ldots, n$$

$$\Downarrow$$

$$\min_{\boldsymbol{w},b} \frac{\|\boldsymbol{w}\|^2}{2} + C \sum_{i=1}^{n} \ell\left(y_i(\boldsymbol{w}'\boldsymbol{x}_i + b)\right)$$

regularization coefficient (positive)

penalty for wrong classifications

Soft Margin

How to define the penalty?

- Cross entropy loss (like logistic regression)
  - Not suitable because SVM doesn't provide a probabilistic output.
- $0-1$ loss
  - Encodes a hard 0 and 1 for incorrect and correct classification respectively.

$$\ell_{0-1}(z) = \begin{cases} 0, & \text{if } z \geq 0 \\ 1, & \text{if } z < 0 \end{cases}$$

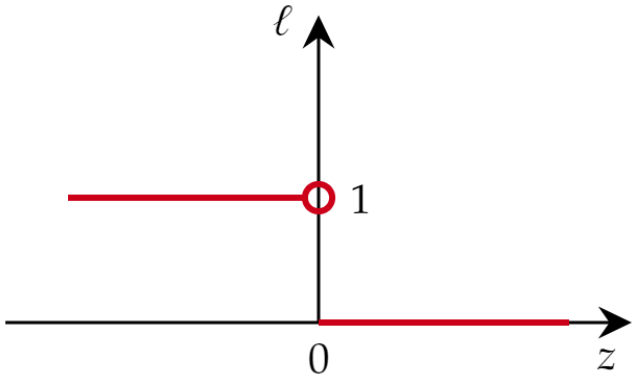
$$z = y_i(\boldsymbol{w}'\boldsymbol{x}_i + b)$$

  - No penalty for correct classifications
  - Penalty of 1 for incorrect classifications

Soft Margin

How to define the penalty?

$$\ell_{0-1}(z) = \begin{cases} 0, & \text{if } z \geq 0 \\ 1, & \text{if } z < 0 \end{cases}$$

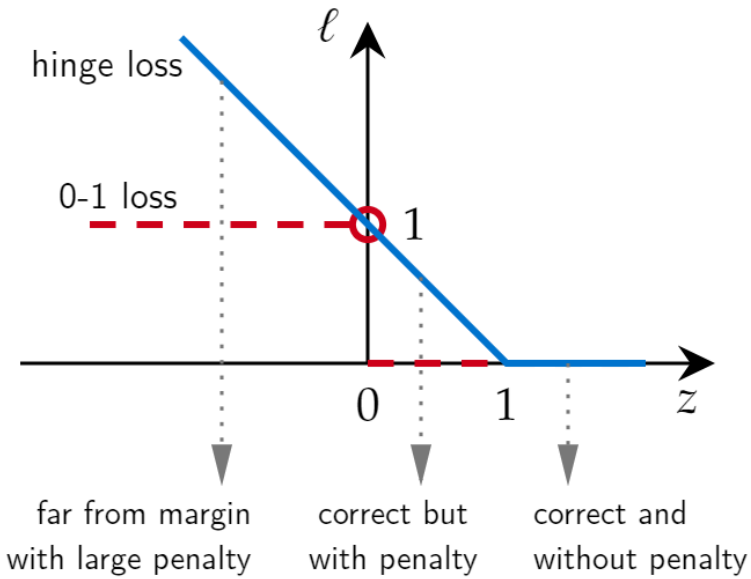


- $0-1$ **loss** is **non-convex**.
- All the incorrect classifications are penalized with 1.

Soft Margin

How to define the penalty?

$$\ell_h(z) = \max(0, 1 - z)$$



- **Hinge loss** is **convex**.
- Samples further from the margin boundary have large penalty.

Soft Margin SVM

The optimization problem is formulated as follows:

$$\min_{\boldsymbol{w},b} \frac{\|\boldsymbol{w}\|^2}{2} + C\sum_{i=1}^{n} \max\left(0, 1 - y_i(\boldsymbol{w}'\boldsymbol{x}_i + b)\right)$$

$$\text{Linear SVM} + \text{Hinge loss} = \text{Soft margin SVM}$$

How to address the non-differentiable objective function?

- Use sub-gradient descent
- Transform the problem to a differentiable one

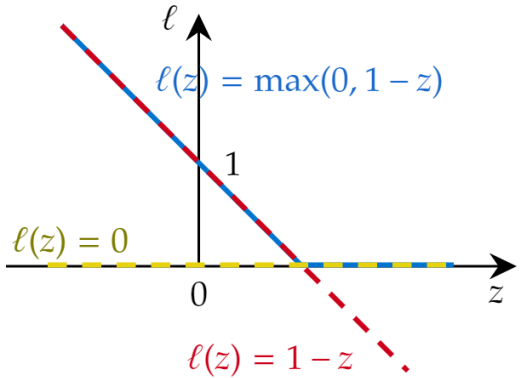
Soft Margin SVM

Let's transform the problem by introducing the slack variable ξ_i:

$$\min_{\boldsymbol{w},b,\boldsymbol{\xi}} \frac{\|\boldsymbol{w}\|^2}{2} + C\sum_{i=1}^{n} \xi_i \qquad \xi_i = \max(0, 1 - y_i(\boldsymbol{w}'\boldsymbol{x}_i + b))$$

$$\text{s.t.} \quad y_i(\boldsymbol{w}'\boldsymbol{x}_i + b) \geq 1 - \xi_i, \quad i = 1, 2, \ldots, n$$

$$\xi_i \geq 0, \quad i = 1, 2, \ldots, n$$



$$\xi_i \geq 1 - z_i$$

$$\xi_i \geq 0$$

Dual Problem of Soft Margin SVM

Again, we need to use Lagrangian dual to deal with complex constraints.

Primal constraints:

1. $1 - \xi_i - y_i(\boldsymbol{w}'\boldsymbol{x}_i + b) \leq 0$
2. $-\xi_i \leq 0$

Dual Variables: α_i for constraint 1, μ_i for constraint 2.

Lagrangian:

$$L(\boldsymbol{w}, b, \boldsymbol{\xi}, \boldsymbol{\alpha}, \boldsymbol{\mu}) = \frac{\|\boldsymbol{w}\|^2}{2} + C\sum_{i=1}^{n}\xi_i + \sum_{i=1}^{n}\alpha_i\left(1 - \xi_i - y_i(\boldsymbol{w}'\boldsymbol{x}_i + b)\right) - \sum_{i=1}^{n}\mu_i\xi_i$$

Dual Problem of Soft Margin SVM

Gradient of Lagrangian:

$$\begin{aligned}
\nabla_{\boldsymbol{w}} L(\boldsymbol{w}^*, b^*, \boldsymbol{\xi}^*, \boldsymbol{\alpha}^*, \boldsymbol{\mu}^*) &= 0 \quad \Leftrightarrow \quad \boldsymbol{w}^* = \sum_{i=1}^{n} \alpha_i^* y_i \boldsymbol{x}_i \\
\nabla_{b} L(\boldsymbol{w}^*, b^*, \boldsymbol{\xi}^*, \boldsymbol{\alpha}^*, \boldsymbol{\mu}^*) &= 0 \quad \Leftrightarrow \quad 0 = \sum_{i=1}^{n} \alpha_i^* y_i \\
\nabla_{\boldsymbol{\xi}} L(\boldsymbol{w}^*, b^*, \boldsymbol{\xi}^*, \boldsymbol{\alpha}^*, \boldsymbol{\mu}^*) &= 0 \quad \Leftrightarrow \quad \mu_i^* = C - \alpha_i^*
\end{aligned}$$

Dual problem:

$$\begin{aligned}
&\max_{\boldsymbol{\alpha}} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \boldsymbol{x}_i' \boldsymbol{x}_j \\
\text{s.t.} \quad &\sum_{i=1}^{n} \alpha_i y_i = 0, \\
&0 \leq \alpha_i \leq C, \quad i = 1, 2, \ldots, n
\end{aligned}$$

Dual Problem of Soft Margin SVM

KKT conditions:

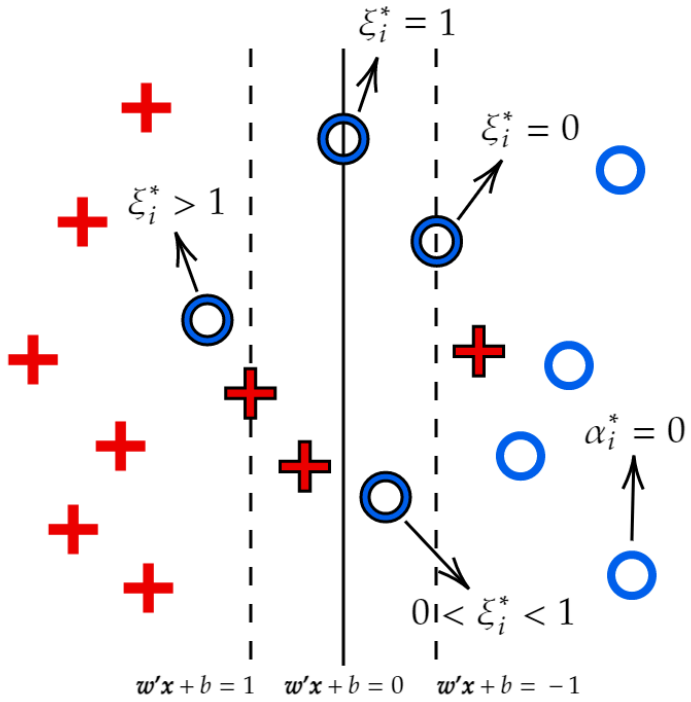
1. *Primal constraints*: $1 - \xi_i^* - y_i(\boldsymbol{w}^{*\prime}\boldsymbol{x}_i + b^*) \leq 0,\ \xi_i^* \geq 0, \quad i = 1, 2, \ldots, n$
2. *Dual constraints*: $\alpha_i^* \geq 0,\ \mu_i^* \geq 0, \quad i = 1, 2, \ldots, n$
3. *Complementary slackness*:

$$\begin{aligned} \alpha_i^* \left(1 - \xi_i^* - y_i(\boldsymbol{w}^{*\prime}\boldsymbol{x}_i + b^*)\right) &= 0, \quad i = 1, 2, \ldots, n \\ \mu_i^* \xi_i^* &= 0, \quad i = 1, 2, \ldots, n \end{aligned}$$

4. *Gradient of Lagrangian*:

$$\begin{aligned} \boldsymbol{w}^* &= \sum_{i=1}^n \alpha_i^* y_i \boldsymbol{x}_i \\ 0 &= \sum_{i=1}^n \alpha_i^* y_i \\ \mu_i^* &= C - \alpha_i^* \end{aligned}$$

Dual Problem of Soft Margin SVM



- $\alpha_i^* = 0, \xi_i^* = 0, z_i^* \geq 1$

  Not a support vector.

  Hinge loss $= 0$.
- $\alpha_i^* \in (0, C), \xi_i^* = 0, z_i^* = 1$

  Is a support vector.

  Hinge loss $= 0$.
- $\alpha_i^* = C, \xi_i^* \geq 0, z_i^* \leq 1$

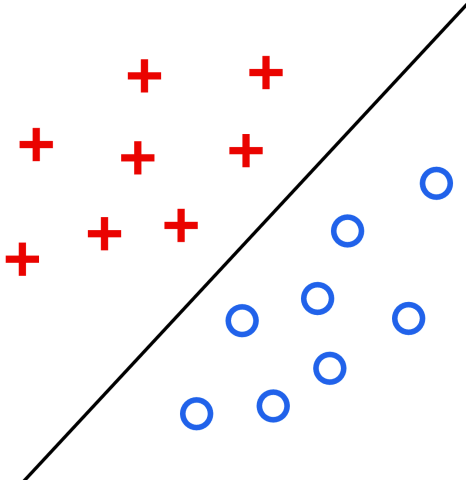
  Is a support vector.

  Hinge loss ≥ 0.

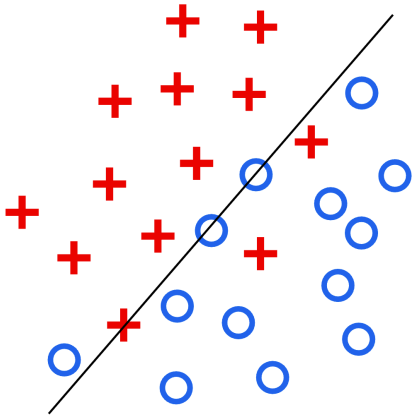
Kernel Trick

Non-Separable Case

Problems that have been solved by hard/soft-margin linear SVM.



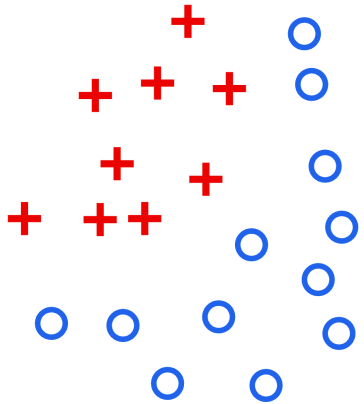
Linearly Separable

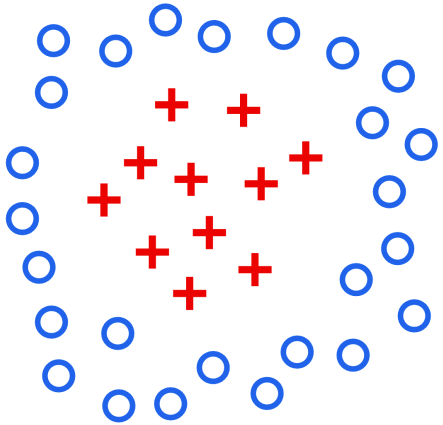


Nearly Linearly Separable

Non-Separable Case

What about these cases?



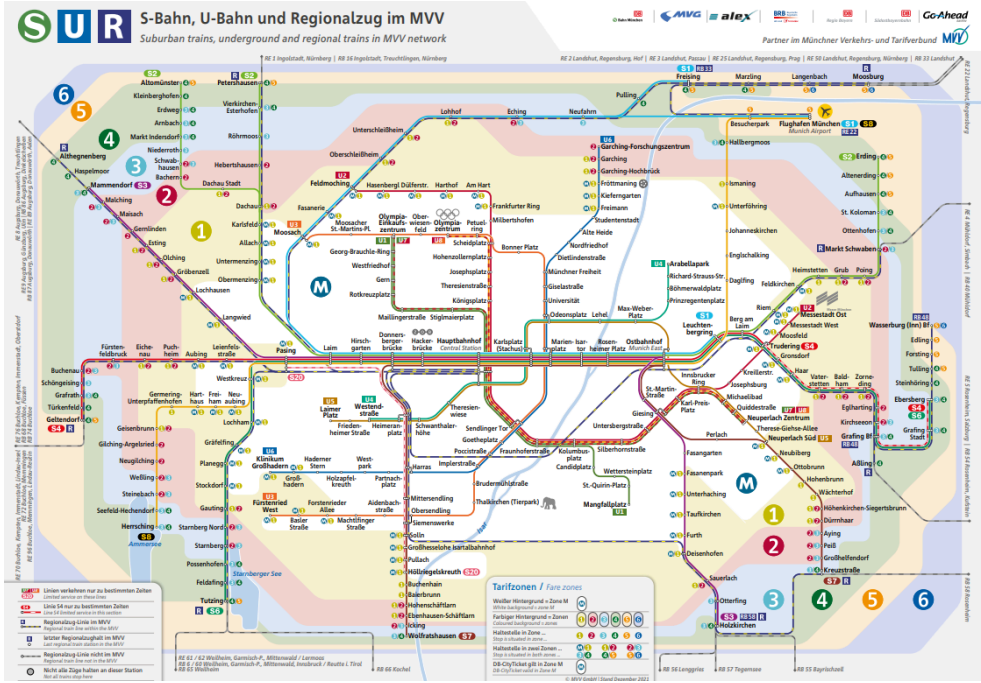


Distance Measure

How could space travel be realized?

Distance Measure

Is distance on map the same as actual travel distance?



Munich Rail Network (Source: MVV-München)

Distance Measure

By distorting the space, we are actually redefining the distance in a space.

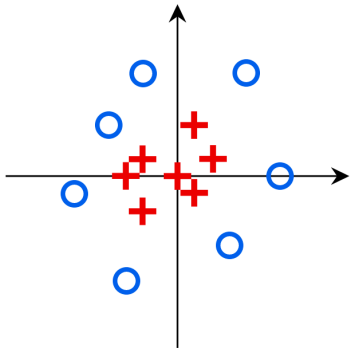
In a two-dimensional Euclidean space, the distance between two points is defined as:

$$d(\boldsymbol{x}, \boldsymbol{y}) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

However, for a generic mathematical space, the definition of distance is not necessarily Euclidean.

$$d(\boldsymbol{x}, \boldsymbol{y}) = f(\boldsymbol{x}, \boldsymbol{y})$$

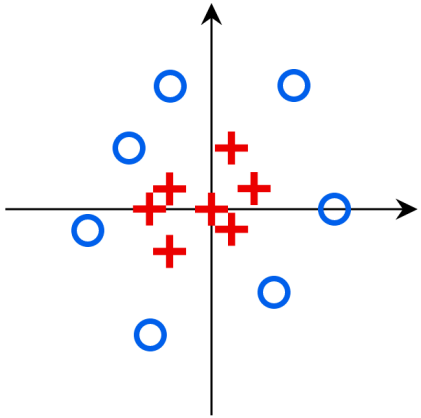
Can we define a new distance measure for the data on the right?

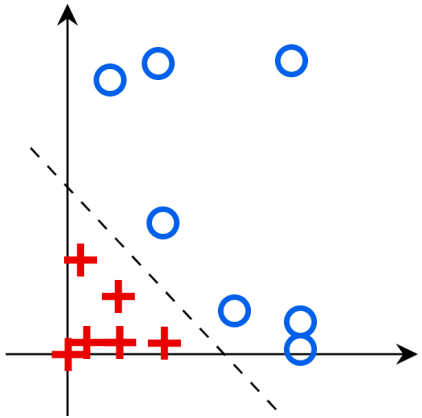


Feature Space Mapping

Build a mapping $\phi(\boldsymbol{x})$ to map features in the original feature space to a new one such that samples of two classes become linearly separable.

$$\phi(\boldsymbol{x}) = \boldsymbol{x}^2$$



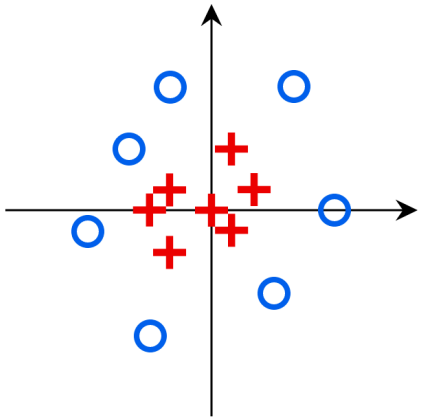


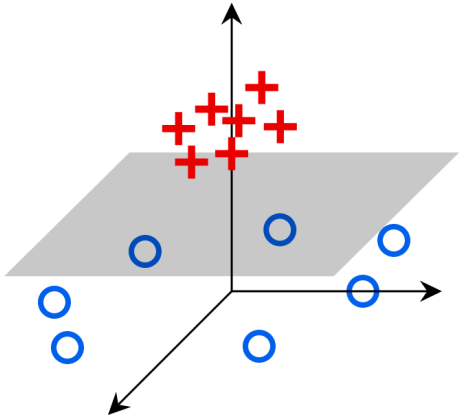
Feature Space Mapping

We can also map the features to a higher dimensional space.

The separating hyperplane in the new space will be:

$$\boldsymbol{w}'\phi(\boldsymbol{x}) + b = 0$$





Nonlinear SVM

Replace the expression of hyperplane in soft-margin linear SVM by the new transformed hyperplane yields:

$$\begin{aligned} &\min_{\boldsymbol{w},b,\boldsymbol{\xi}} \frac{\|\boldsymbol{w}\|^2}{2} + C\sum_{i=1}^{n}\xi_i \\ \text{s.t.} \quad & y_i\left(\boldsymbol{w}'\phi(\boldsymbol{x}_i) + b\right) \geq 1-\xi_i, \quad i = 1,2,\ldots,n \\ & \xi_i \geq 0, \quad i = 1,2,\ldots,n \end{aligned}$$

Dual Problem of Nonlinear SVM

Following the same derivation as linear SVM, we can obtain the dual problem of nonlinear SVM:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \phi(\boldsymbol{x}_i)' \phi(\boldsymbol{x}_j)$$

$$\phi(\boldsymbol{x}_i)' \phi(\boldsymbol{x}_j) = \langle \phi(\boldsymbol{x}_i), \phi(\boldsymbol{x}_j) \rangle$$

$$\text{s.t.} \quad \sum_{i=1}^{n} \alpha_i y_i = 0,$$

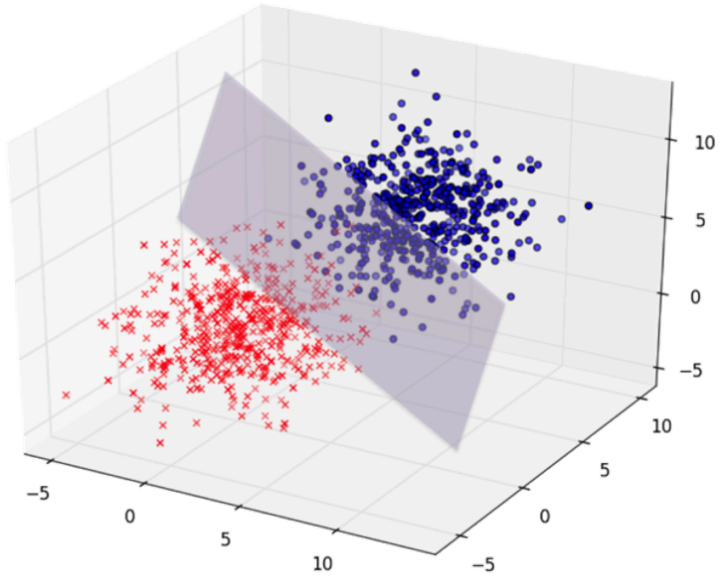
$$0 \leq \alpha_i \leq C, \quad i = 1, 2, \ldots, n$$

The only difference is that we apply an mapping on $\boldsymbol{x}_i$ and $\boldsymbol{x}_j$ before computing their inner product.

Kernel Trick

The issue of the mapping $\phi(\boldsymbol{x})$:

- It's hard to find an appropriate $\phi(\boldsymbol{x})$ for the data.
- Computational cost can be huge when dealing with high-dimensional features.



Kernel Trick

Let's get back the dual problem of nonlinear SVM:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{n} \alpha_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \alpha_i \alpha_j y_i y_j \phi(\boldsymbol{x}_i)' \phi(\boldsymbol{x}_j)$$

$$\text{s.t.} \quad \sum_{i=1}^{n} \alpha_i y_i = 0,$$

$$0 \leq \alpha_i \leq C, \quad i = 1, 2, \ldots, n$$

Do we really need to know $\phi(\boldsymbol{x})$ explicitly?

Kernel Trick

Use a surrogate function $\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) = \phi(\boldsymbol{x}_i)'\phi(\boldsymbol{x}_j)$.

The separating hyperplane will be transformed to:

$$\boldsymbol{w}'\phi(\boldsymbol{x}) + b = 0 \Rightarrow \left(\sum_{i=1}^{n} \alpha_i^* y_i \phi(\boldsymbol{x}_i)\right)' \phi(\boldsymbol{x}) + b = 0$$

$$\Rightarrow \sum_{i=1}^{n} \alpha_i^* y_i \kappa(\boldsymbol{x}_i, \boldsymbol{x}) + b = 0$$

Kernel trick helps us to skip the procedure of *mapping* and *computing inner product* by directly defining the inner product in the new space.

Kernel Trick

Kernel Function

If there exists a mapping $\phi : \mathcal{X} \to \mathcal{H}$, for any $\boldsymbol{x}_1, \boldsymbol{x}_2 \in \mathcal{X}$, the kernel function $\kappa : \mathcal{X} \times \mathcal{X} \to \mathbb{R}$ is given by:

$$\kappa(\boldsymbol{x}_1, \boldsymbol{x}_2) = \phi(\boldsymbol{x}_1)'\phi(\boldsymbol{x}_2) = \langle \phi(\boldsymbol{x}_1), \phi(\boldsymbol{x}_2) \rangle_{\mathcal{H}}$$

*The kernel function redefines the **similarity** rather than **distance** between samples.*

Common Kernel Functions

Commonly used kernel functions include:

- Linear kernel
- Radial basis function (RBF) kernel (aka Gaussian kernel)
- Polynomial kernel
- Matérn kernel
- Sigmoid kernel (aka hyperbolic tangent kernel)

Common Kernel Functions

Linear kernel : No transformation is performed.

$$\kappa_{\text{linear}}(\boldsymbol{x}_1, \boldsymbol{x}_2) = \boldsymbol{x}_1' \boldsymbol{x}_2$$

Polynomial kernel :

$$\kappa_{\text{poly},d}(\boldsymbol{x}_1, \boldsymbol{x}_2) = (\boldsymbol{x}_1' \boldsymbol{x}_2 + c)^d$$

where $d \in \mathbb{Z}_+, c \in \mathbb{R}$ are the hyperparameters of polynomial kernel.

Common Kernel Functions

RBF kernel : RBF kernel is the most widely used kernel.

$$\kappa_{\text{RBF}}(\boldsymbol{x}_1, \boldsymbol{x}_2) = \exp\left(-\frac{(\boldsymbol{x}_1 - \boldsymbol{x}_2)'(\boldsymbol{x}_1 - \boldsymbol{x}_2)}{2\sigma^2}\right)$$

where $\sigma > 0$ is the hyperparameter of RBF kernel.

Anisotropic RBF kernel :

$$\kappa_{\text{poly},d}(\boldsymbol{x}_1, \boldsymbol{x}_2) = \exp\left(-\frac{1}{2}(\boldsymbol{x}_1 - \boldsymbol{x}_2)'\Sigma(\boldsymbol{x}_1 - \boldsymbol{x}_2)\right)$$

where $\Sigma = \text{diag}(\sigma_1^2, \sigma_2^2, \ldots, \sigma_m^2)$ is the hyperparameter of anisotropic RBF kernel that defines a different σ for each dimension of feature.

Common Kernel Functions

Rational quadratic kernel :

$$\kappa_{\mathrm{rq}}(\boldsymbol{x}_1, \boldsymbol{x}_2) = \left(1 + \frac{(\boldsymbol{x}_1 - \boldsymbol{x}_2)'(\boldsymbol{x}_1 - \boldsymbol{x}_2)}{2\alpha l^2}\right)^{-\alpha}$$

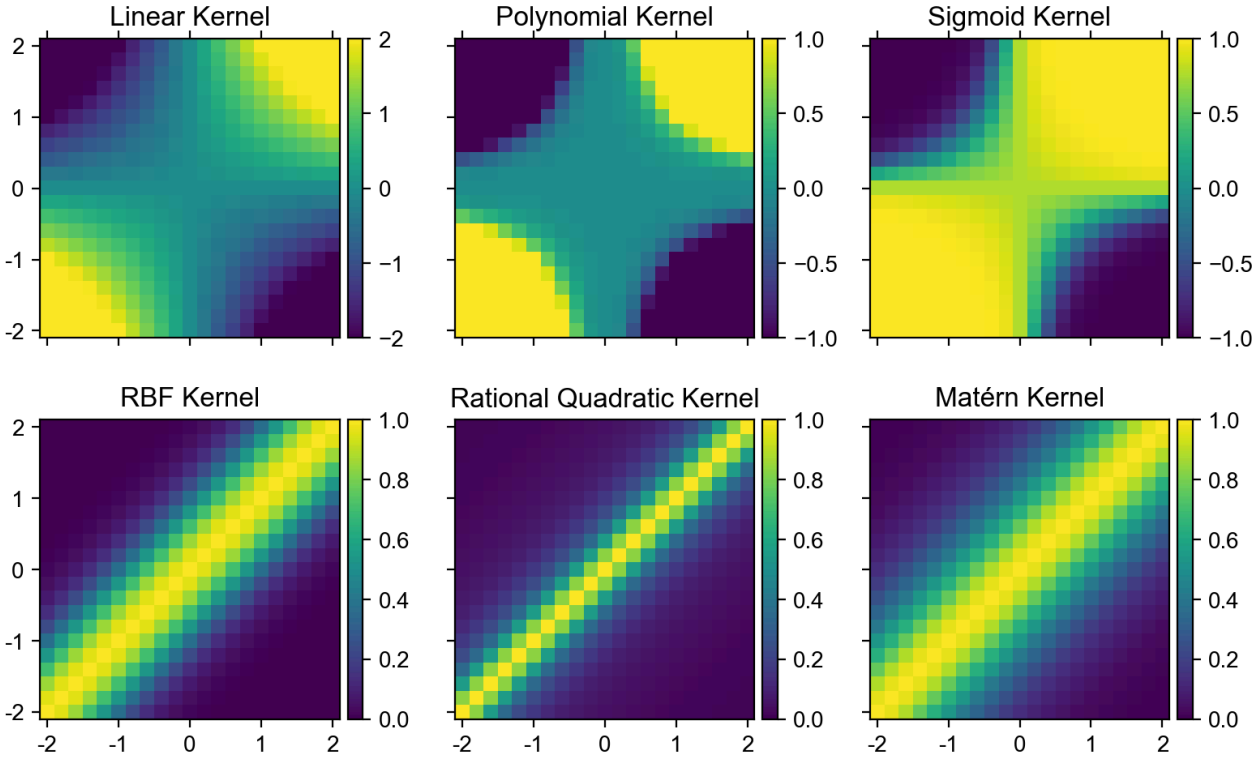
where $\alpha > 0, l > 0$ are the hyperparameters of rational quadratic kernel.

Sigmoid kernel : Sigmoid kernel comes from neural network.

$$\kappa_{\mathrm{sigmoid}}(\boldsymbol{x}_1, \boldsymbol{x}_2) = \tanh(\alpha \boldsymbol{x}_1' \boldsymbol{x}_2 + \beta)$$

where $\alpha > 0, \beta < 0$ are the hyperparameters of sigmoid kernel.

Common Kernel Functions



Common Kernel Functions

Note: Not all functions are eligible as kernel functions (i.e., a valid mapping ϕ exists).

Mercer's condition : Define a Gram matrix (aka kernel matrix) of a **symmetric** function κ with respect to **any** $\boldsymbol{x}_1, \boldsymbol{x}_2, \ldots, \boldsymbol{x}_n$:

$$\boldsymbol{K} = \begin{pmatrix} \kappa(\boldsymbol{x}_1, \boldsymbol{x}_1) & \kappa(\boldsymbol{x}_1, \boldsymbol{x}_2) & \cdots & \kappa(\boldsymbol{x}_1, \boldsymbol{x}_n) \\ \kappa(\boldsymbol{x}_2, \boldsymbol{x}_1) & \kappa(\boldsymbol{x}_2, \boldsymbol{x}_2) & \cdots & \kappa(\boldsymbol{x}_2, \boldsymbol{x}_n) \\ \vdots & \vdots & \ddots & \vdots \\ \kappa(\boldsymbol{x}_n, \boldsymbol{x}_1) & \kappa(\boldsymbol{x}_n, \boldsymbol{x}_2) & \cdots & \kappa(\boldsymbol{x}_n, \boldsymbol{x}_n) \end{pmatrix}$$

A semi-positive definite Gram matrix ensures this function to be a valid kernel function.

Exceptions: Some kernel functions like sigmoid kernel do not always satisfy Mercer's condition.

Common Kernel Functions

Composition of kernels

Compositions of Mercer kernel functions still satisfy Mercer's condition.

$$\kappa(\boldsymbol{x}_1, \boldsymbol{x}_2) = \begin{cases} c\kappa_1(\boldsymbol{x}_1, \boldsymbol{x}_2) \\ \kappa_1(\boldsymbol{x}_1, \boldsymbol{x}_2) + \kappa_2(\boldsymbol{x}_1, \boldsymbol{x}_2) \\ \kappa_1(\boldsymbol{x}_1, \boldsymbol{x}_2) \cdot \kappa_2(\boldsymbol{x}_1, \boldsymbol{x}_2) \end{cases}$$

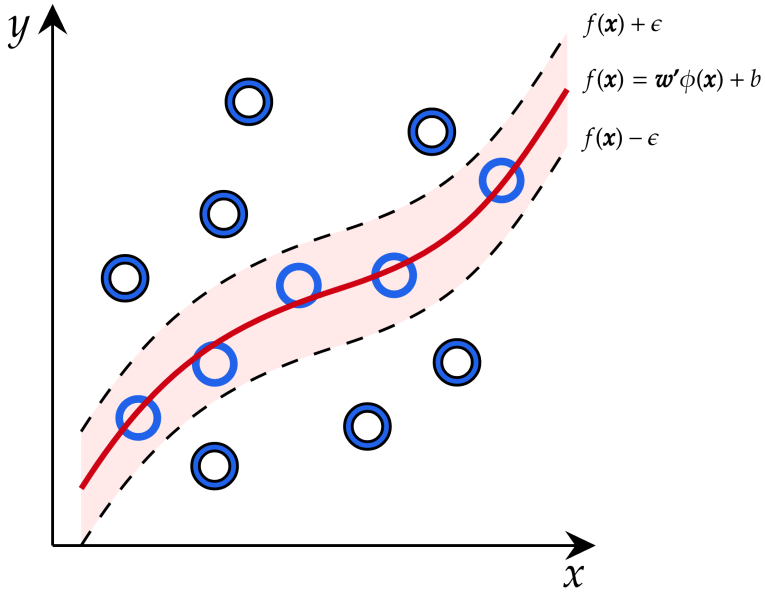
Support Vector Regression

Support Vector Regression

Although SVM was originally designed for solving classification problems, it can also be modified for regression problems.

Basic idea: Create a ϵ-tube around $f(\boldsymbol{x})$, within which the samples are assumed to be correctly predicted.

- Ideally, the goal is to find a function that deviates from the actual target values by a value no greater than ϵ.
- Samples outside this ϵ-tube incur a loss.



Support Vector Regression

The generic definition of SVR is:

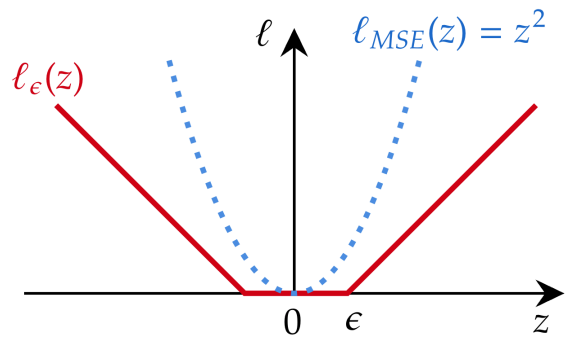
$$\min_{\boldsymbol{w},b} \frac{\|w\|^2}{2} + C \sum_{i=1}^{N} \ell_\epsilon \left(f(\boldsymbol{x}_i) - y_i \right)$$

ϵ-insensitive loss

regularization coefficient (positive)

$$\ell_\epsilon(z) = \begin{cases} 0, & \text{if } |z| \leq \epsilon; \\ |z| - \epsilon, & \text{otherwise} \end{cases}$$

$$z_i = f(\boldsymbol{x}_i) - y_i$$



Is there any connection with ridge regression?

Support Vector Regression

Similar to what we did to soft-margin linear SVM, the problem can be transformed by introducing slack variables ξ_i^+, ξ_i^-.

$$
\begin{aligned}
\min_{\boldsymbol{w},b,\boldsymbol{\xi}^+,\boldsymbol{\xi}^-} \quad & \frac{\|w\|^2}{2} + C\sum\nolimits_{i=1}^{N}(\xi_i^+ + \xi_i^-) \\
\text{s.t.} \quad & f(\boldsymbol{x}_i) - y_i \leq \epsilon + \xi_i^+, \quad i = 1, 2, \ldots, n \\
& y_i - f(\boldsymbol{x}_i) \leq \epsilon + \xi_i^-, \quad i = 1, 2, \ldots, n \\
& \xi_i^+, \xi_i^- \geq 0, \quad i = 1, 2, \ldots, n
\end{aligned}
$$

Hints: Rewrite the ϵ-insensitive loss as the sum of two sub-functions:

$$
\ell_\epsilon(z) = \ell_\epsilon^+(z) + \ell_\epsilon^-(z)
$$

$$
\begin{aligned}
\ell_\epsilon^+(z) &= \max(0, z - \epsilon) \\
\ell_\epsilon^-(z) &= \max(0, -z - \epsilon)
\end{aligned}
$$

Dual Problem of SVR

Primal constraints:

1. $f(\boldsymbol{x}_i) - y_i \leq \epsilon + \xi_i^+$
2. $y_i - f(\boldsymbol{x}_i) \leq \epsilon + \xi_i^-$
3. $\xi_i^+ \geq 0$
4. $\xi_i^- \geq 0$

Dual Variables: $\alpha_i^+, \alpha_i^-, \mu_i^+, \mu_i^-$ for constraints 1–4.

Lagrangian: $L(\boldsymbol{w}, b, \boldsymbol{\xi}^+, \boldsymbol{\xi}^-, \boldsymbol{\alpha}^+, \boldsymbol{\alpha}^-, \boldsymbol{\mu}^+, \boldsymbol{\mu}^-)$

$$
\begin{aligned}
&= \frac{\|\boldsymbol{w}\|^2}{2} + C \sum_{i=1}^n (\xi_i^+ + \xi_i^-) - \sum_{i=1}^n \mu_i^+ \xi_i^+ - \sum_{i=1}^n \mu_i^- \xi_i^- \\
&\quad + \sum_{i=1}^n \alpha_i^+ \left(f(\boldsymbol{x}_i) - y_i - \epsilon - \xi_i^+ \right) \\
&\quad + \sum_{i=1}^n \alpha_i^- \left(y_i - f(\boldsymbol{x}_i) - \epsilon - \xi_i^- \right)
\end{aligned}
$$

Dual Problem of SVR

Gradient of Lagrangian:

$$\begin{aligned}
\nabla_{\boldsymbol{w}} L = 0 &\Leftrightarrow \boldsymbol{w}^* = \sum_{i=1}^{n} (\alpha_i^{-*} - \alpha_i^{+*}) \phi(\boldsymbol{x}_i) \\
\nabla_b L = 0 &\Leftrightarrow 0 = \sum_{i=1}^{n} (\alpha_i^{-*} - \alpha_i^{+*}) \\
\nabla_{\boldsymbol{\xi}^+} L = 0 &\Leftrightarrow \mu_i^{+*} = C - \alpha_i^{+*} \\
\nabla_{\boldsymbol{\xi}^-} L = 0 &\Leftrightarrow \mu_i^{-*} = C - \alpha_i^{-*}
\end{aligned}$$

The prediction for new data points can be expressed as:

$$\begin{aligned}
f(\boldsymbol{x}) = \boldsymbol{w}^{*\prime} \phi(\boldsymbol{x}) + b &= \left(\sum_{i=1}^{n} (\alpha_i^{-*} - \alpha_i^{+*}) \phi(\boldsymbol{x}_i) \right)' \phi(\boldsymbol{x}) + b \\
&= \sum_{i=1}^{n} (\alpha_i^{-*} - \alpha_i^{+*}) \kappa(\boldsymbol{x}, \boldsymbol{x}_i) + b
\end{aligned}$$

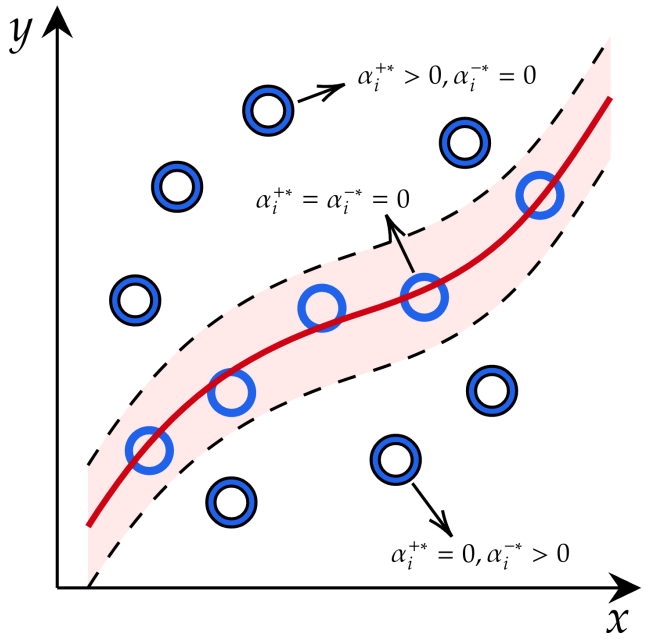
Dual Problem of SVR

Substitute into Lagrangian to get the dual problem of SVR.

Dual problem:

$$\begin{aligned}
\max_{\boldsymbol{\alpha}^+,\boldsymbol{\alpha}^-} & \sum_{i=1}^{n}(\alpha_i^- - \alpha_i^+)y_i - \epsilon\sum_{i=1}^{n}(\alpha_i^- + \alpha_i^+) \\
& - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n}(\alpha_i^- - \alpha_i^+)(\alpha_j^- - \alpha_j^+)\kappa(\boldsymbol{x}_i, \boldsymbol{x}_j) \\
\text{s.t.} \quad & \sum_{i=1}^{n}(\alpha_i^- - \alpha_i^+) = 0, \\
& 0 \leq \alpha_i^+, \alpha_i^- \leq C, \quad i = 1, 2, \ldots, n
\end{aligned}$$

Dual Problem of SVR



- $\alpha_i^{+*} > 0, \alpha_i^{-*} = 0$

  Is a support vector.
- $\alpha_i^{+*} = 0, \alpha_i^{-*} > 0$

  Is a support vector.
- $\alpha_i^{+*} = \alpha_i^{-*} = 0$

  Not a support vector.

Prediction is the weighted sum of support vectors.

$$f(\boldsymbol{x}) = \sum_{i=1}^{n} (\alpha_i^{-*} - \alpha_i^{+*}) \kappa(\boldsymbol{x}, \boldsymbol{x}_i) + b$$

Hyperparameter Optimization and Gaussian Process

Hyperparameter

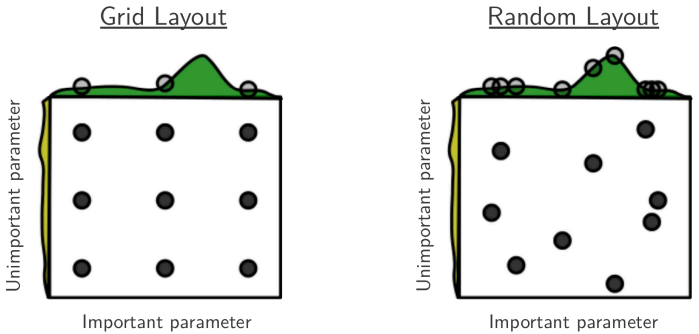
In addition to model parameters, statistical learning models often come with many hyperparameters, which cannot be optimized during the learning process.

- **Model structure**
  - Maximum depth of decision tree
  - Number of trees in random forest
  - Number of layers of neural network
- **Optimization**
  - Learning rate
  - Batch size
- **Regularization**
  - L_2 penalty
  - C of SVM
- **Preprocessing**
  - Data scaling
- ...

Why can't we directly optimize hyperparameters?

Grid Search & Random Search

- **Grid search** exhaustively evaluates all possible hyperparameter combinations to find the best-performing model configuration.
- **Random search** evaluates a random subset of hyperparameter combinations to efficiently explore the search space.

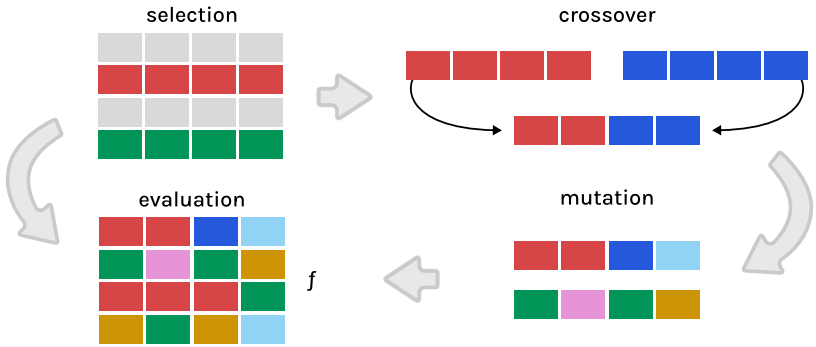


Source: Bergstra & Bengio (2012) Random Search for Hyper-Parameter Optimization

Swarm and Evolutionary Computation

- **Swarm intelligence** algorithms works by mimicking the collective behavior of natural systems like bird flocks or fish schools.
  - Particle swarm optimization, artificial bee colony algorithm, …
- **Evolutionary computation** algorithms works by simulating the process of natural evolution, using mechanisms such as selection, mutation, and crossover.
  - Genetic algorithm, differential evolution, …

- High evaluation cost
- Slow convergence



Bayesian Optimization

Hyperparameter tuning involves a black-box optimization problem:

$$x^* \in \arg\max_{x \in \mathcal{X}} f(x)$$

- No access to gradient information
- Trial and error is necessary to evaluate hyperparameter

?

?

?

Bayesian Optimization

0€
1€
0€
1€
0€
1€

3€
0€
4€
1€
2€
3€
0€
1€

2€
1€

- Sequential search, unlike parallel search, incorporates previous observations.

Bayesian Optimization

$$\hat{\mu}_1^{(15)} = 2/5 = 0.4$$
$$n_1^{(15)} = 5$$

$$\hat{\mu}_2^{(15)} = 14/8 = 1.75$$
$$n_2^{(15)} = 8$$

$$\hat{\mu}_3^{(15)} = 3/2 = 1.5$$
$$n_3^{(15)} = 2$$

- To improve the searching efficiency, we have to balance **exploitation** (observation) and **exploration** (uncertainty).

Bayesian optimization (BO) works by building a surrogate function of the unknown objective, and manage exploitation-exploration trade-off using Bayesian methods.

Bayesian Optimization

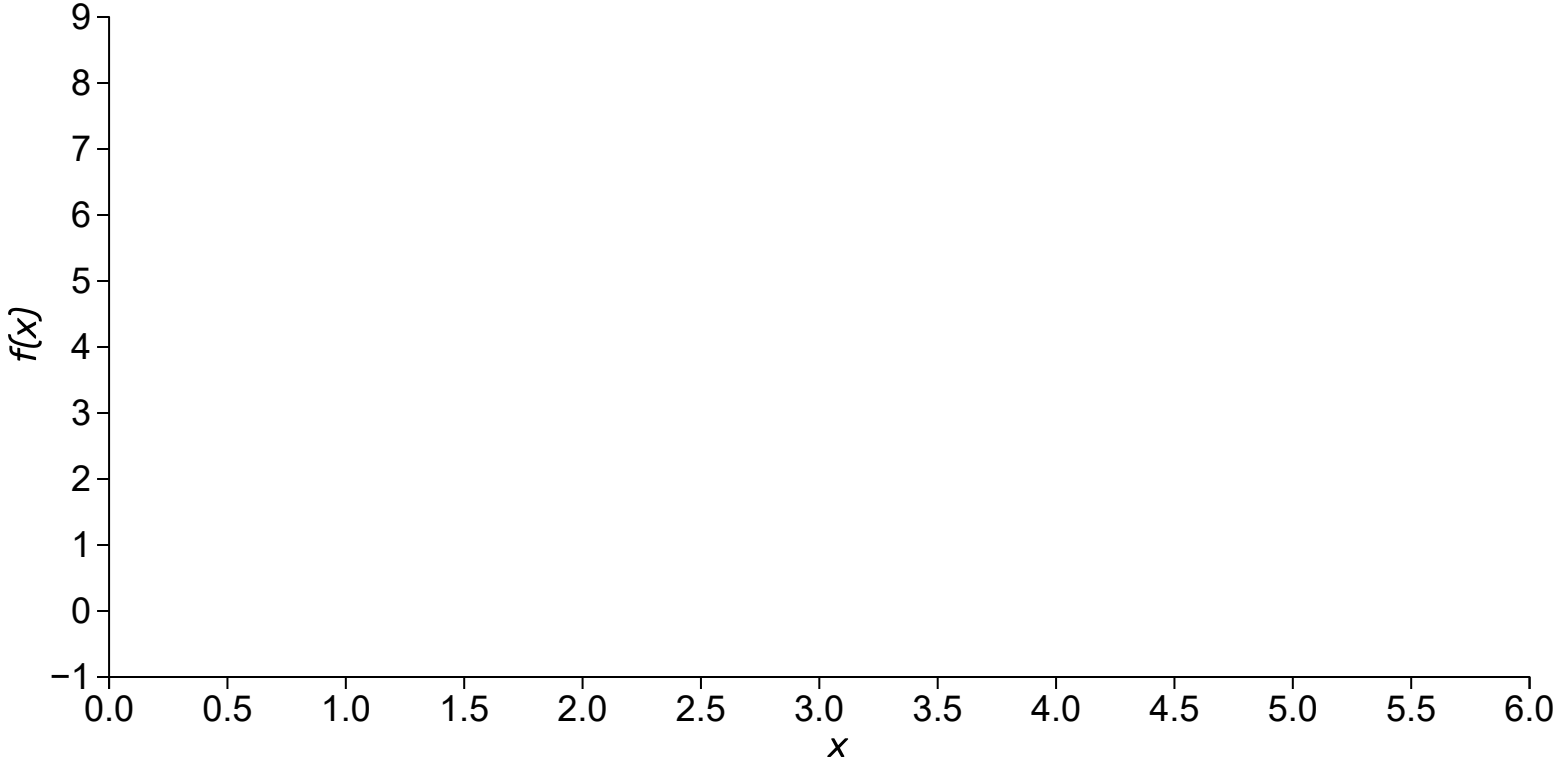
Consider a black-box optimization problem:

$$x^* \in \arg\max_{x \in \mathcal{X}} f(x)$$

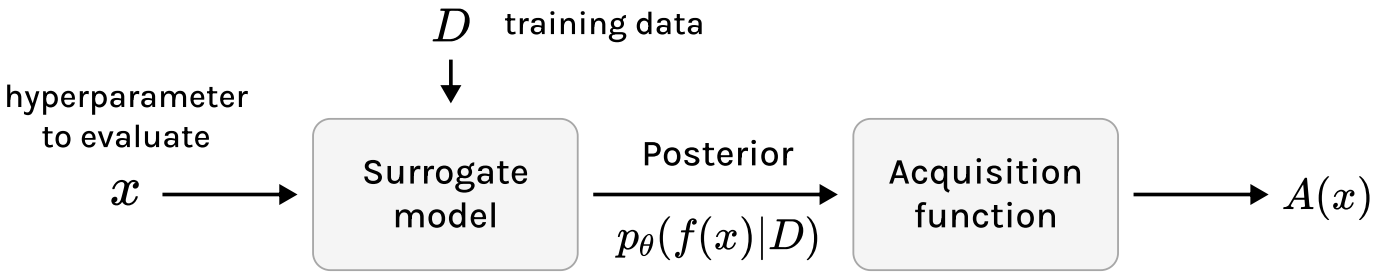
Properties:

- The input x is in $\mathbb{R}^d$, where d is **not too large**, typically $d < 20$.
- The feasible set $\mathcal{X}$ is a simple set, in which it is easy to assess membership.
- The true objective function f is a **black-box** and **expensive to evaluate**. It is also continuous and lacks known special structure like concavity or linearity.
- Focusing on finding **a global optimum** rather than a local one.

Bayesian Optimization



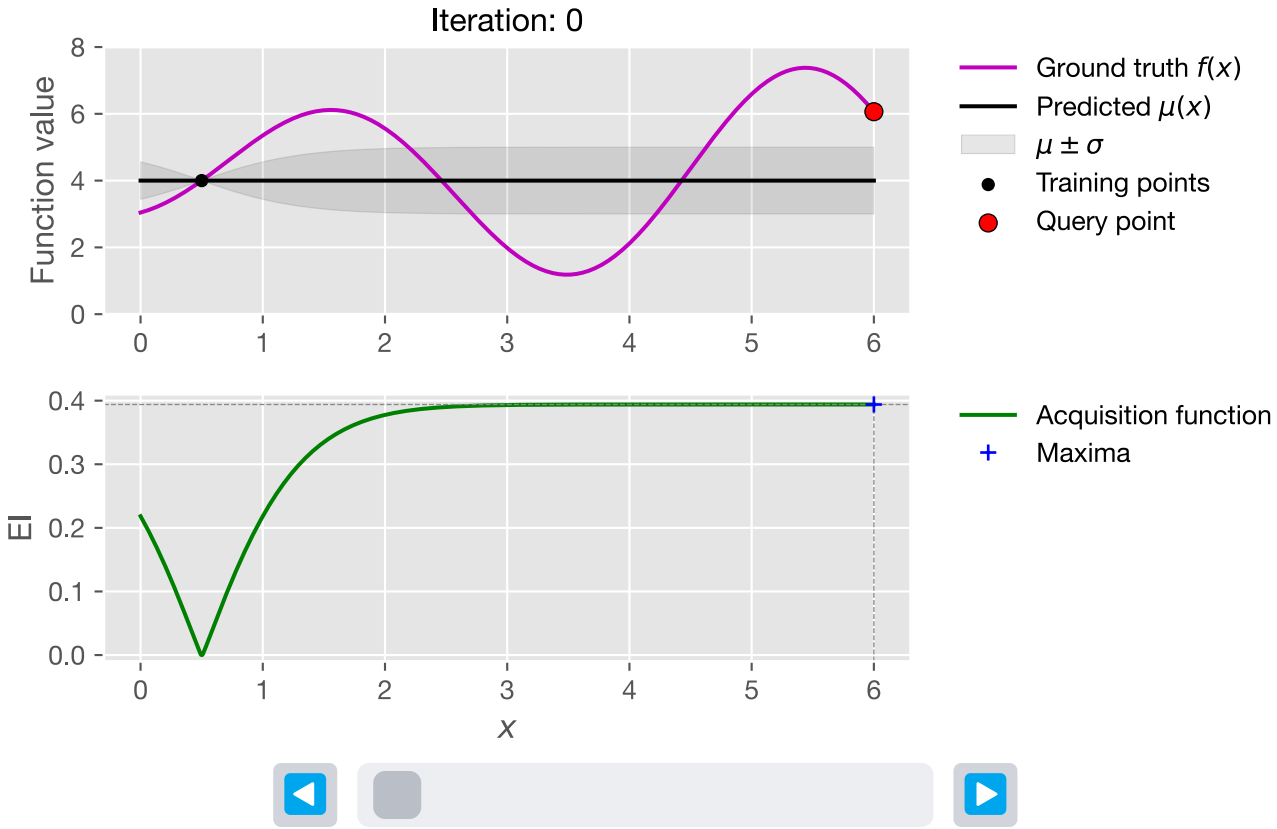
Bayesian Optimization



Main components of Bayesian optimization:

- **Bayesian statistical model** :
  - Approximate the objective function
- **Acquisition function** :
  - Determine the next point to sample

Bayesian Optimization



Bayesian Optimization

- **Gaussian process regression (GPR)** is the most common choice of the Bayesian statistical model in BO. It is based on **GP**, which is a stochastic process where any finite set of random variables follows a multivariate Gaussian distribution.
  - Finite dimensional: multivariate Gaussian distribution, characterized by a **mean vector** $\boldsymbol{\mu}$ and a **covariance matrix** $\boldsymbol{K}$.

$$\boldsymbol{X} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{K})$$

  - Infinite dimensional: Gaussian process, characterized by a **mean function** $m(\cdot)$ and a **kernel/covariance function** $\kappa(\cdot, \cdot)$.

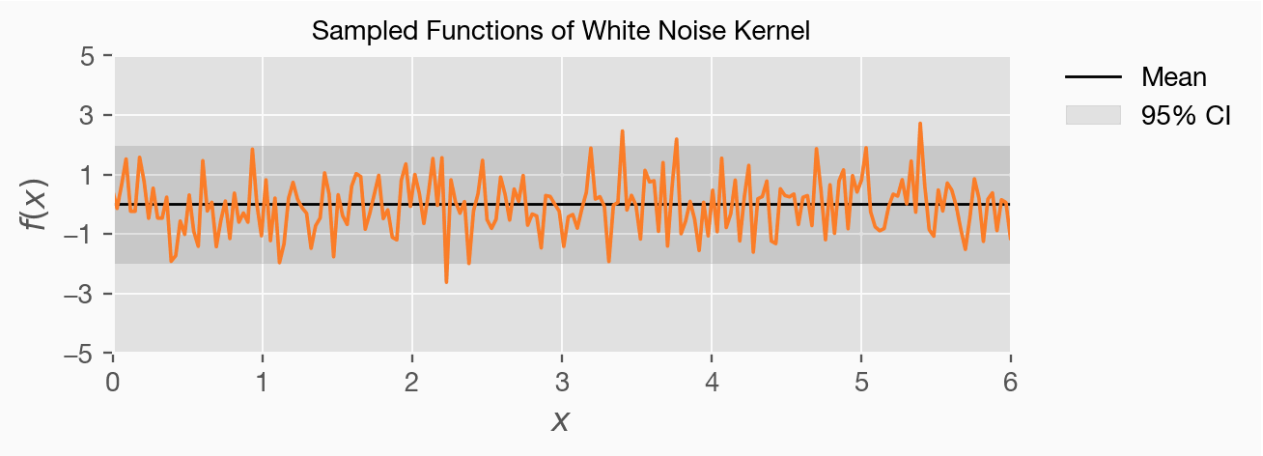
$$f(\boldsymbol{x}) \sim \mathcal{GP}(m(\boldsymbol{x}), \kappa(\boldsymbol{x}, \cdot))$$

Bayesian Optimization

The kernel function is a **prior** of GP.

The **white kernel** assumes independence between different points.

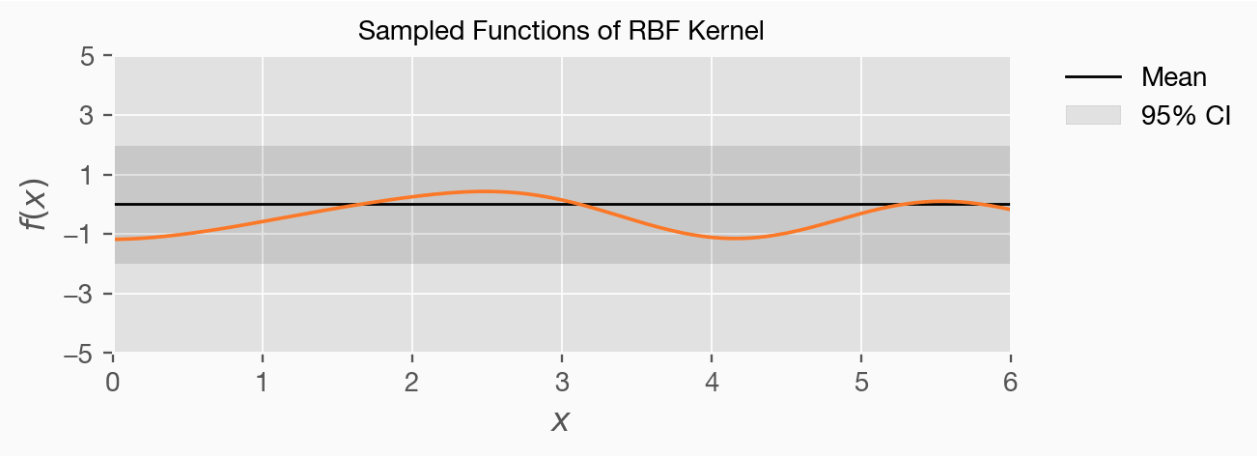
$$\kappa_{\text{white}}(x, z) = \boldsymbol{I}_x$$



Bayesian Optimization

The **RBF kernel** imposes a smooth prior on the sampled functions, parameterized by a length scale coefficient l.

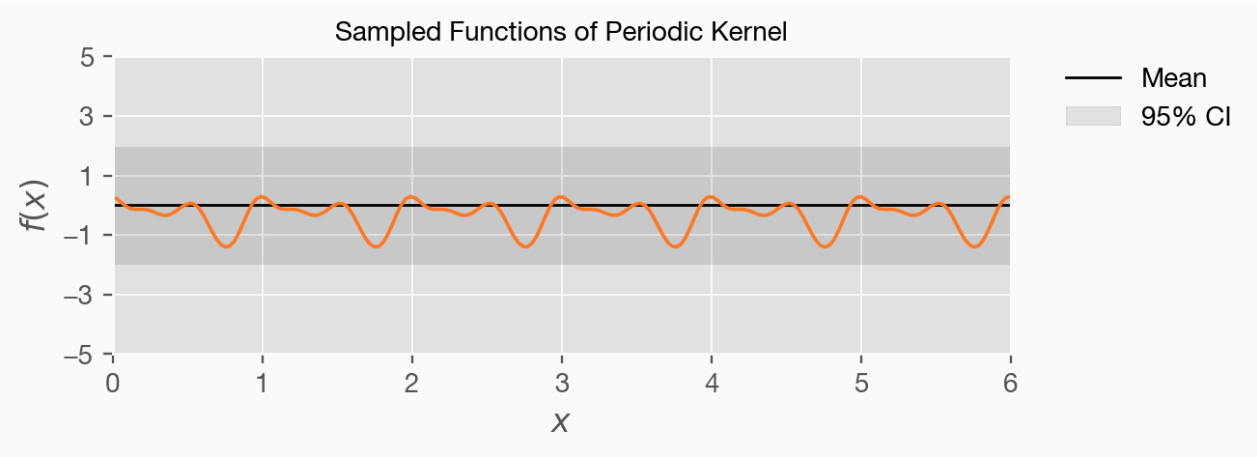
$$\kappa_{\mathrm{rbf}}(x, z) = \exp\left(-\frac{\|x - z\|^2}{2l^2}\right)$$



Bayesian Optimization

The **periodic kernel** imposes a smooth periodic prior on the sampled functions, parameterized by a length scale coefficient l and periodicity p.

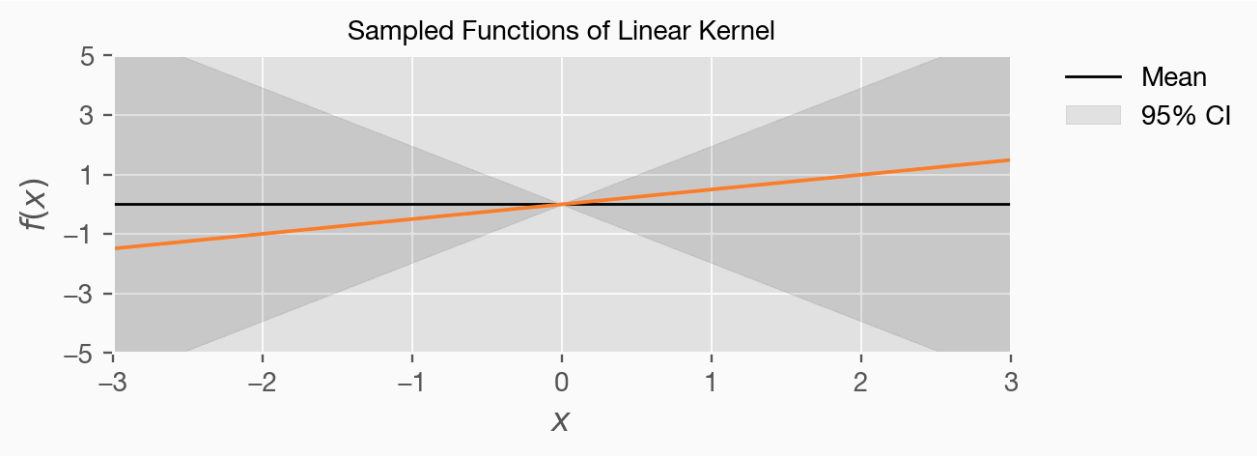
$$\kappa_{\text{periodic}}(x, z) = \exp\left(-\frac{2}{l^2}\sin^2\left(\frac{\|x - z\|}{p/\pi}\right)\right)$$



Bayesian Optimization

The **linear kernel** imposes a linear non-stationary prior on the sampled functions, parameterized by an offset c.

$$\kappa_{\text{linear}}(x, z) = (x - c)'(z - c)$$



Bayesian Optimization

Given the GP with prior mean and kernel function: $f(\boldsymbol{x}) \sim \mathcal{GP}(m(\boldsymbol{x}), \kappa(\boldsymbol{x}, \cdot))$, the observed data follows the multivariate Gaussian distribution:

$$\boldsymbol{Y} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{K})$$

where

$$\boldsymbol{\mu} = \begin{pmatrix} m(x_1) \\ m(x_2) \\ \vdots \\ m(x_n) \end{pmatrix}, \boldsymbol{K} = \begin{pmatrix} \kappa(x_1, x_1) & \kappa(x_1, x_2) & \dots & \kappa(x_1, x_n) \\ \kappa(x_2, x_1) & \kappa(x_2, x_2) & \dots & \kappa(x_2, x_n) \\ \vdots & \vdots & \ddots & \vdots \\ \kappa(x_n, x_1) & \kappa(x_n, x_2) & \dots & \kappa(x_n, x_n) \end{pmatrix}$$

A common choice of prior mean is constant zero or constant mean.

Bayesian Optimization

For the random variable Y_* corresponding to an arbitrary point x_*, it follows a **joint multivariate Gaussian distribution with observed data**:

$$\begin{pmatrix} \boldsymbol{Y} \\ Y_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu} \\ \mu_* \end{pmatrix}, \begin{pmatrix} \boldsymbol{K} & \boldsymbol{\kappa}_* \\ \boldsymbol{\kappa}_*' & \kappa_{**} \end{pmatrix} \right)$$

The inference of GPR is to compute the **posterior distribution** of GP, which can be computed by the marginalizing the joint distribution above on the observed data:

$$\begin{aligned} Y_* | \boldsymbol{Y}, \boldsymbol{X}, X_* &\sim \mathcal{N}(m_*, \Sigma_*) \\ m_* &= \mu_* + \boldsymbol{\kappa}_*' \boldsymbol{K}^{-1} (\boldsymbol{y} - \boldsymbol{\mu}) \\ \Sigma_* &= \kappa_{**} - \boldsymbol{\kappa}_*' \boldsymbol{K}^{-1} \boldsymbol{\kappa}_* \end{aligned}$$

where $\boldsymbol{\kappa}_* = (\kappa(x_*, x_1), \ldots, \kappa(x_*, x_n))'$, $\kappa_{**} = \kappa(x_*, x_*)$.

Bayesian Optimization

- **Handling observational noises**:
  - Apply white noise on the kernel matrix of observed data:

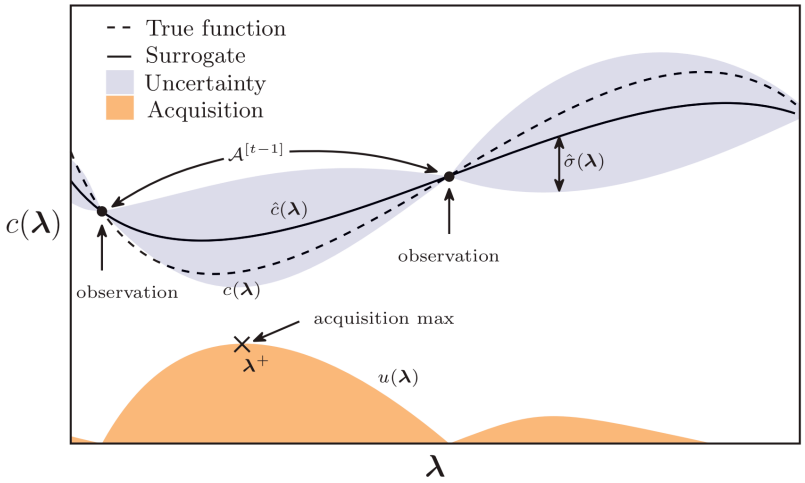
$$\begin{pmatrix} \boldsymbol{Y} \\ Y_* \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} \boldsymbol{\mu} \\ \mu_* \end{pmatrix}, \begin{pmatrix} \boldsymbol{K} + \alpha \boldsymbol{I} & \boldsymbol{\kappa}_* \\ \boldsymbol{\kappa}_*' & \kappa_{**} \end{pmatrix} \right)$$

- **Tuning parameters of kernel functions**:
  - Maximize the log marginal likelihood:

$$\max_{\theta} \log p(\boldsymbol{Y}|\boldsymbol{X}, \theta) = -\frac{1}{2} \underbrace{\boldsymbol{Y}^\top (\boldsymbol{K} + \alpha \boldsymbol{I})^{-1} \boldsymbol{Y}}_{\text{fitting}} - \frac{1}{2} \underbrace{\log |\boldsymbol{K} + \alpha \boldsymbol{I}|}_{\text{model complexity}} - \frac{n}{2} \log 2\pi$$

Bayesian Optimization

- **Acquisition function** is a function of the posterior distribution to trade-off between exploration and exploitation.
  - To better **exploit** what we have known, we want to choose a point with high **posterior mean**.
  - To better **explore** what we don't know, we want to choose a point with high **uncertainty**.



Source: Bischl et al. (2021) Hyperparameter Optimization: Foundations, Algorithms, Best Practices and Open Challenges

Bayesian Optimization

Expected Improvement (EI) is the expectation of the improvement brought by observing a new point.

$$A_{\mathrm{EI}}(x) = \mathbb{E}\left(\max(0, f(x) - f(x^+))\right)$$

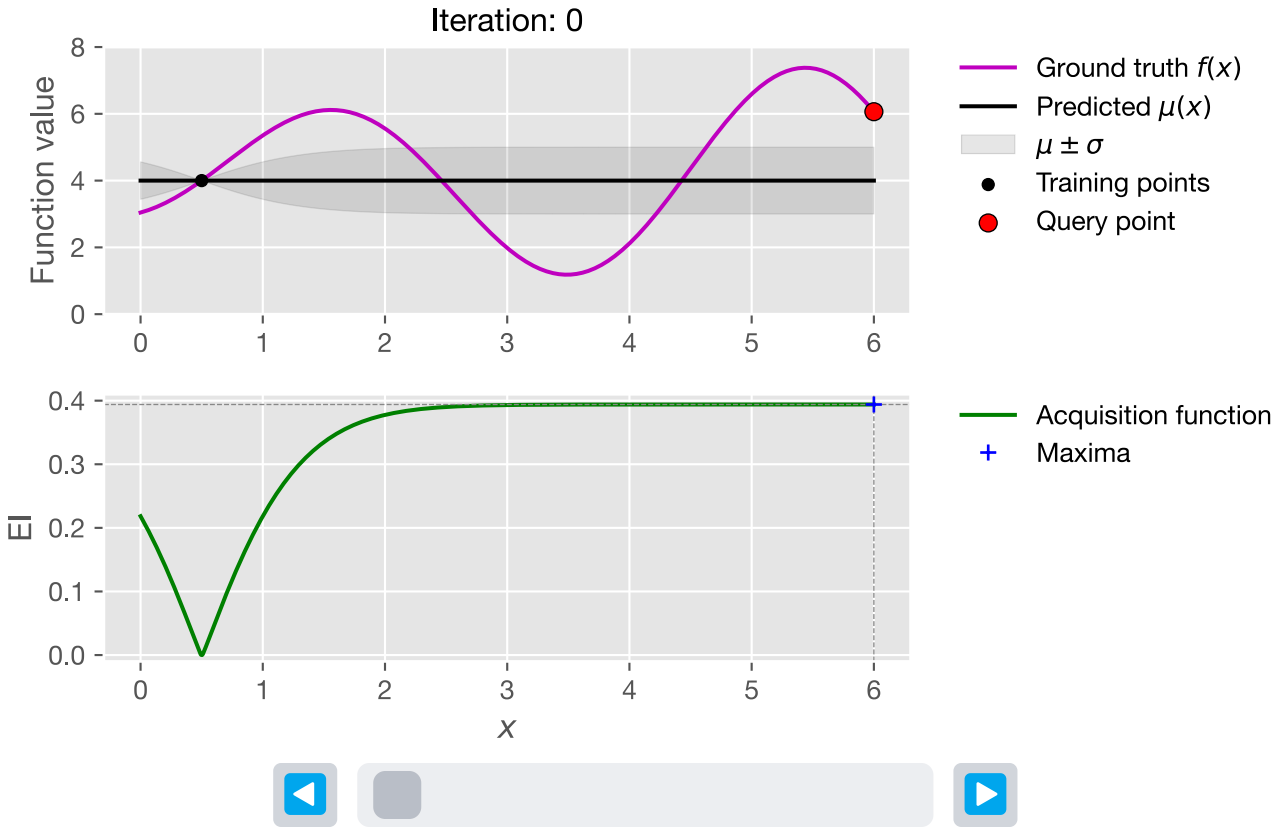
There is an analytical expression:

$$A_{\mathrm{EI}}(x) = \begin{cases} (\mu_t(x) - f(x^+) - \epsilon)\,\Phi(Z) + \sigma_t(x)\phi(Z), & \text{if } \sigma_t(x) > 0 \\ 0, & \text{if } \sigma_t(x) = 0 \end{cases}$$

$$Z = \frac{\mu_t(x) - f(x^+) - \epsilon}{\sigma_t(x)}$$

where $\Phi(\cdot)$ and $\phi(\cdot)$ are the cdf and pdf of standard Gaussian distribution.

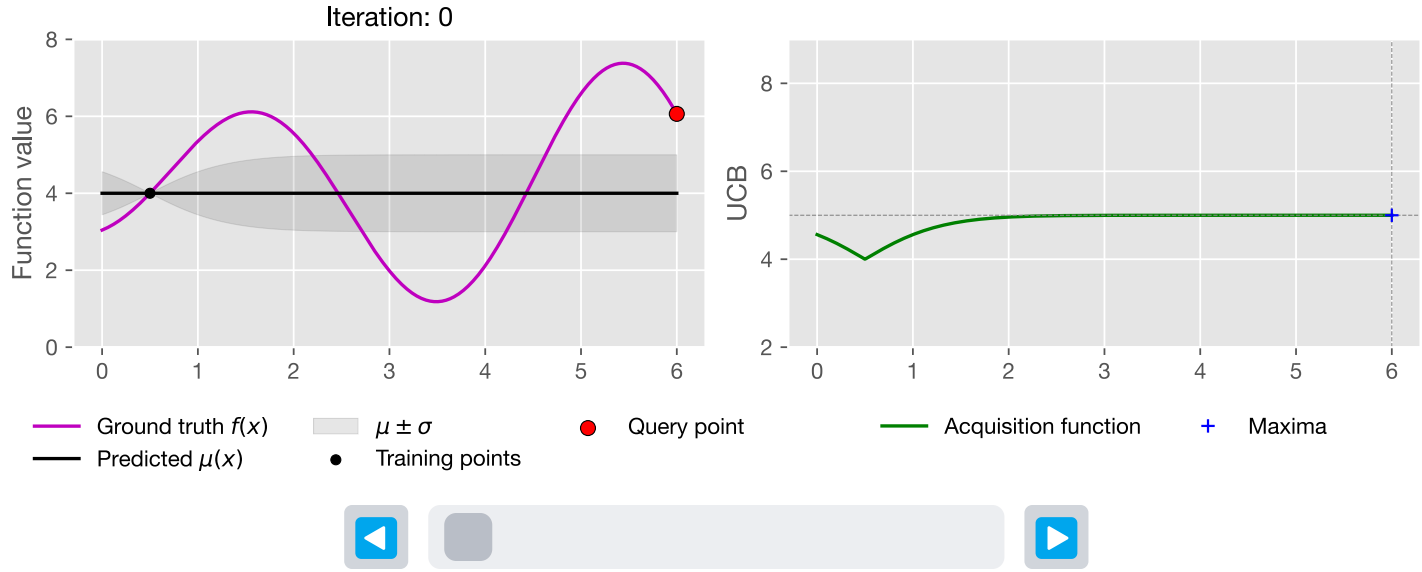
Bayesian Optimization



Bayesian Optimization

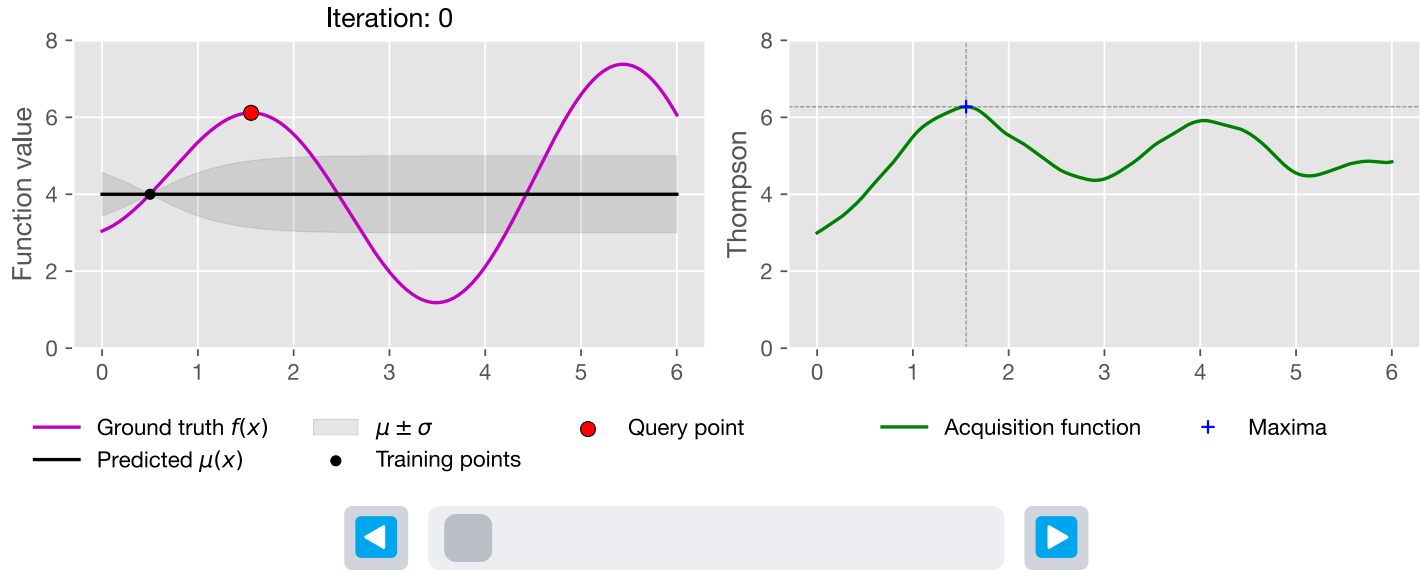
Upper confidence bound (UCB) is a linear combination of the posterior mean and standard deviation.

$$A_{\mathrm{UCB}}(x) = \mu(x) + \sqrt{\beta}\sigma(x)$$



Bayesian Optimization

Thompson sampling selects the next point by iteratively sampling from the posterior distribution.



Soban Lone
soban.lone@tum.de

Technical University of Munich
TUM School of Engineering and Design
Chair of Transportation Systems Engineering

Munich, 05/06/2026